Programming & Data Structures

GATE CSE 2027 · complete formula, rule, method & trap sheet

63 reference boxes · **9** chapters ·

C Programming Fundamentals | Pointers, Arrays, Strings & Memory | Functions & Recursion | Arrays, Stacks & Queues | Linked Lists | Trees, BST & AVL | Heaps, Hashing & Graph Representation | Problem → Standard Method Playbook | Last-minute one-liners

1 C Programming Fundamentals

COMMON TRAP

Scope of this assumption

These sizes are a convention for the examples and practice questions in this hub, **not a property of C**. The C standard only guarantees `sizeof(char) == 1`, minimum value ranges (`int` ≥ 16 bits, `long` ≥ 32 bits, `long long` ≥ 64 bits) and the ordering `short ≤ int ≤ long ≤ long long`. In a real GATE question, **use the machine assumptions the question gives** (e.g. "4-byte int", "8-byte pointers") and do not carry the numbers below into other problems.

Data types and sizes

REFERENCE TABLE

Type	Size assumed in this section	Notes
char	1 byte	sizeof(char) is always 1; may be signed or unsigned (implementation-defined)
short / int	2 / 4 bytes	int is the "natural" word size
long / long long	8 (Linux 64-bit) / 8 bytes	long is 4 bytes on some platforms — read the question
float / double	4 / 8 bytes	IEEE 754 single / double
pointer	8 bytes (64-bit)	size does not depend on the type pointed to

NOTES

Unsigned arithmetic wraps modulo 2^n. **Signed** overflow is **undefined behaviour**. In mixed signed/unsigned expressions the signed operand is converted to unsigned.

Operators, precedence and evaluation

RULES & FACTS

- **Integer division truncates toward zero** (C99): `7/2 = 3`, `-7/2 = -3`, `7/-2 = -3`. The remainder takes the sign of the dividend: `-7 % 2 = -1`. Always `(a/b)*b + a%b == a`.
- Precedence (high → low, simplified): `() [] -> .` → unary `! ~ ++ -- * & sizeof` → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment → `,`.
- Relational operators are left-associative and yield 0 or 1, so `5 > 3 > 1` is `(5 > 3) > 1` = `1 > 1` = **0**.
- **Short-circuit**: in `a && b` b is evaluated only if a is true; in `a || b` only if a is false. So `if (i++ && j++)` with i = 0, j = 0 leaves **i = 1, j = 0**.
- Prefix `++i` increments then yields the new value; postfix `i++` yields the old value then increments.
- **Undefined behaviour**

  Modifying a variable twice between sequence points — `i = i++ + ++i`, `a[i] = i++`, `printf("%d %d", i++, i)` — is undefined or unspecified. GATE usually avoids it; if an option says "undefined/compiler dependent", it is often right.

WORKED EXAMPLE

Worked numerical — integer and unsigned semantics

```
unsigned char c = 200;
c = c + 100;                    // 300 mod 256 = 44
→ prints 44
printf("%d", -1 < 1u);          // -1 converted to unsigned
= 4294967295 → comparison false → 0
printf("%d", -7/2 + -7%2);      // -3 + (-1) = -4
```

Output of the three prints: **44, 0, −4** (verified by compiling and running).

WORKED EXAMPLE

Worked numerical — switch fall-through and loops

```
int k = 3;
switch (k) { case 1: printf("one ");  case 3: printf("three
");  case 4: printf("four "); break;  default: printf("def
"); }
// prints:  three four       (no break after case 3, so
execution falls into case 4)

for (i = 0; i < 5; i++) { if (i == 2) continue; if (i == 4)
break; s += i; }
// s = 0 + 1 + 3 = 4, and i ends at 4
```

Bitwise operations

KEY POINT

Handy identities

```
x & (x-1)   clears the lowest set bit
x & -x       isolates the lowest set bit
x << k       = x · 2^k     (no overflow)
x >> k       = ⌊x / 2^k⌋   (unsigned)
x ^ x = 0,   x ^ 0 = x
a ^= b; b ^= a; a ^= b;   swaps without temp
```

KEY POINT

Counting set bits (Kernighan)

```
while (x) { x &= x - 1; count++; }
// 0x2D = 101101  → 4 iterations
// 40 & 39 = 32  (101000 → 100000)
```

Storage classes and scope

REFERENCE TABLE

Class	Lifetime / scope
auto (default for locals)	created on entry to the block, destroyed on exit; stack; uninitialised = garbage
register	hint to keep in a register; you cannot take its address
static (local)	lives for the whole program, initialised **once** (to 0 by default), keeps its value between calls; scope is still the block
static (file level)	internal linkage — visible only inside that file
extern	declares a global defined elsewhere; globals default to 0

WORKED EXAMPLE

Worked numerical — static variable

```
int cnt() { static int n = 0; return ++n; }
cnt(); cnt(); printf("%d", cnt());      // static n persists
→ 3
```

The preprocessor

NOTES

Macros are **textual substitution** before compilation — no type checking, no evaluation of arguments. Always parenthesise arguments and the whole body.

WORKED EXAMPLE

Worked numerical — macro pitfalls

```
#define SQ(x) x*x
SQ(2+3)       →  2+3*2+3 = 11           (not 25)
100 / SQ(5)   →  100/5*5 = 20*5 = 100   (not 4)
#define SQ2(x) ((x)*(x))   →  SQ2(2+3) = 25   ✔
```

2 Pointers, Arrays, Strings & Memory

Pointer arithmetic

RULES & FACTS

- For `T *p`, `p + k` advances by **k · sizeof(T)** bytes. `p - q` (same array) gives the number of **elements** between them, not bytes.
- `a[i]` is defined as `*(a + i)`, so `i[a]` works too: for `int arr[] = {1,2,3,4}`, `2[arr] = 3`.
- The array name **decays** to a pointer to its first element in most expressions — but `sizeof(a)` gives the whole array (20 bytes for `int a[5]`), while for **a function parameter** `int a[]` is really `int *a`, so `sizeof(a)` there is 8.
- An array name is not an lvalue: `a = p;` is illegal, `a++` is illegal.

KEY POINT

Pre/postfix with pointers

```
*p++    value at p, then p moves  (postfix binds tighter than *)
(*p)++  increments the VALUE
*++p    moves p first, then reads
++*p    increments the value, yields new value
```

KEY POINT

Dereference and address

```
int x = 5, *p = &x, **pp = &p;
*p = 7;     // x is 7
**pp = 9;   // x is 9
*(&x) == x
```

WORKED EXAMPLE

Worked numerical — traversing with a pointer

```
int a[] = {10, 20, 30, 40, 50};  int *p = a;
int v1 = *p++;     // v1 = 10, p now points to a[1]
int v2 = *p;       // v2 = 20
int v3 = ++*p;     // a[1] becomes 21, v3 = 21
// p - a = 1, and a[1] is now 21         → v1, v2, v3 = 10, 20, 21
```

Two-dimensional arrays

NOTES

C stores 2-D arrays in **row-major order**. For `int m[R][C]`: `&m[i][j] = base + (i·C + j)·sizeof(int)`, and `m[i][j] == *(*(m + i) + j)`. Here `m + 1` advances a whole row (C elements), whereas `*m + 1` advances one element.

WORKED EXAMPLE

Worked numerical

`int m[3][4]`: `&m[2][1] − &m[0][0] = 2·4 + 1 = 9` elements. `sizeof(m) = 48`, and `sizeof(m)/sizeof(m[0]) = 3` rows.

Call by value and pointers

NOTES

C passes **everything by value**. A function receives a copy, so swapping two ints requires passing their addresses:

FORMULA / WORKING

```
void swapv(int a, int b) { int t = a; a = b; b = t; }      // swaps the copies only
void swapp(int *a, int *b) { int t = *a; *a = *b; *b = t; } // swaps the caller's variables
// u = 3, w = 4:  swapv(u, w) → still 3 4;   swapp(&u, &w) → now 4 3
```

Strings

RULES & FACTS

- A string is a `char` array ending with `'\0'`. `strlen` does not count the terminator, `sizeof` of an array does.
- `char s[] = "GATE";` → a writable array: **sizeof(s) = 5**, **strlen(s) = 4**. `char *t = "GATE";` → a pointer to a read-only literal: `sizeof(t) = 8`, and modifying `t[0]` is undefined behaviour.
- Comparing strings with `==` compares addresses; use `strcmp`. `strcpy` needs a destination of at least strlen+1 bytes.

Structures, unions and padding

NOTES

Each member is aligned to its own size; the whole struct is padded to a multiple of its **largest alignment**. Member order matters. A **union** shares one block of memory; its size is the largest member, rounded up to the alignment.

WORKED EXAMPLE

Worked numerical — sizeof with padding (verified with gcc)

```
struct A { char c; int i; char d; };        c(1) + pad(3) + i(4) + d(1) + pad(3)   = 12
struct B { int i; char c; char d; };        i(4) + c(1) + d(1) + pad(2)            = 8
struct C { char c; double d; int i; };      c(1) + pad(7) + d(8) + i(4) + pad(4)   = 24
union  U { int i; char c[6]; };             max(4, 6) = 6, rounded to alignment 4   = 8
```

Reordering members from largest to smallest alignment usually minimises padding.

Dynamic memory

REFERENCE TABLE

Function	Behaviour
malloc(n)	n uninitialised bytes from the **heap**; NULL on failure
calloc(k, n)	k·n bytes, **zero-initialised**
realloc(p, n)	resize; may move the block (old pointer becomes invalid)
free(p)	returns the block; `free(NULL)` is safe; freeing twice is undefined

RULES & FACTS

- **Memory leak**: allocated memory that is no longer reachable and never freed.
- **Dangling pointer**: pointer to memory that was freed, or to a local variable that has gone out of scope (returning `&local` from a function).
- **Wild/uninitialised pointer**: never assigned; dereferencing it is undefined. Set pointers to NULL after free.
- Memory layout: **code** (text), **static/global data**, **heap** (grows up), **stack** (grows down).

Function pointers

FORMULA / WORKING

```
int (*op)(int, int) = add;   // pointer to a function returning int taking two ints
int r = op(2, 3);            // calls add(2, 3);   used for callbacks, qsort's comparator
```

3 Functions & Recursion

How a call works

NOTES

Each call pushes an **activation record** (stack frame) holding the parameters, the return address, saved registers and local variables; return pops it. Recursion works because every invocation has its own frame. **Stack depth** = number of active (not yet returned) calls — infinite recursion ends in stack overflow.

RULES & FACTS

- Parameters are passed **by value** (copies). Locals are re-created on every call; **static** locals and globals are shared across all invocations.
- The order in which operands of `+`, function arguments, etc. are evaluated is **unspecified** in C — avoid questions that depend on it (for example, an expression that reads a global that a recursive call in the same expression modifies).
- **Tail recursion**: the recursive call is the very last action (`return f(n-1, acc*n);`); a compiler can reuse the frame, turning it into a loop with O(1) stack. `return n * f(n-1);` is **not** tail recursive (a multiplication follows the call).

Tracing technique

RULES & FACTS

1. Write the call tree. Mark what is printed **before** the recursive calls (pre-order) and **after** them (post-order).
2. Evaluate from the base case upward; keep local variables separate per call.
3. For counting problems write a recurrence for the number of calls and solve it.

WORKED EXAMPLE

Worked numerical — print between two recursive calls

```
void ruler(int n) { if (n == 0) return;  ruler(n-1);
printf("%d ", n);  ruler(n-1); }
ruler(3):   ruler(2) prints  1 2 1;  then 3;  then ruler(2)
prints 1 2 1
Output:  1 2 1 3 1 2 1

Calls: C(0) = 1,  C(n) = 2·C(n−1) + 1  →  C(n) = 2^(n+1) − 1  →
C(3) = 15 calls
Number of values printed = 2^n − 1 = 7
```

Classic recursive functions

REFERENCE TABLE

Function	Recurrence	Result
factorial	n! = n · (n−1)!	n calls deep → O(n) time, O(n) stack
gcd(a, b)	gcd(b, a mod b); gcd(a, 0) = a	gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) = **6**
Fibonacci (naive)	F(n) = F(n−1) + F(n−2)	exponential time Θ(φ^n); stack depth only n
Tower of Hanoi	H(n) = 2H(n−1) + 1	2^n − 1 moves: 3 disks → 7, 10 disks → **1023**
Binary search (recursive)	T(n) = T(n/2) + 1	O(log n) time and stack
McCarthy 91	f(n) = n−10 if n > 100, else f(f(n+11))	f(n) = 91 for every n ≤ 100: f(50) = f(99) = **91**

WORKED EXAMPLE

Worked numerical — number of calls in naive Fibonacci

Let C(n) = calls made by `fib(n)` : C(0) = C(1) = 1 and C(n) = C(n−1) + C(n−2) + 1.

```
C(2) = 3   C(3) = 5   C(4) = 9   C(5) = 15   C(6) = 25
In general C(n) = 2·F(n+1) − 1.  fib(5) returns 5 and makes
15 calls (counted by running the program).
```

Many of these calls recompute the same value — the reason dynamic programming and memoisation turn the time into O(n).

GATE FAVOURITE

GATE angle

Three things to check every time: (1) **what is printed before vs after** the recursive call; (2) **static / global** state shared across calls; (3) the **base case** — a missing or unreachable base case means infinite recursion. Also be ready to give the space complexity as the **maximum depth** of the recursion, not the number of calls.

4 Arrays, Stacks & Queues

Arrays

REFERENCE TABLE

Operation	Unsorted array	Sorted array
Access by index	O(1)	O(1)
Search	O(n)	O(log n) (binary search)
Insert / delete at a position	O(n) (shift elements); O(1) at the end	O(n)

NOTES

Address of element i of a 1-D array = `base + (i − lower)·w`; 2-D addressing is as in the pointer section. Sparse matrices are stored as triples (row, col, value) or in CSR form to save space.

Stack (LIFO)

NOTES

Operations **push, pop, peek, isEmpty** all take O(1). Uses: function calls, undo, expression evaluation, balanced-bracket checking, DFS, backtracking.

RULES & FACTS

- **Two stacks in one array** of size n: start one at index 0 growing up, the other at n−1 growing down; overflow only when they meet — the whole array is usable.
- **Stack permutations**: if 1, 2, …, n are pushed in order and popped at any time, the number of possible pop sequences is the Catalan number C(n) = (1/(n+1))·C(2n, n): n = 3 → **5** (only 3 1 2 is impossible), n = 4 → **14**.

Infix → postfix with a stack

NOTES

Scan left to right. Operands go straight to the output. For an operator, pop operators of **higher or equal precedence** (for left-associative operators; for the right-associative `^` only strictly higher) to the output, then push it. `(` is pushed; `)` pops until `(`. At the end pop everything.

WORKED EXAMPLE

Worked numerical — conversions

```
A + B * C - D / E        →  A B C * + D E / -          (max stack size 2)
A * (B + C) / D - E      →  A B C + * D / E -          (max stack size 3, counting the parenthesis)
(A + B) * (C - D)        →  A B + C D - *
```

Postfix evaluation

NOTES

Operands are pushed; an operator pops two values (right operand first), applies the operation and pushes the result. The stack depth needed is the maximum number of pending operands.

WORKED EXAMPLE

Worked numerical

`5 6 2 + * 12 4 / -`:

```
push 5, 6, 2     [5 6 2]
+                [5 8]
*                [40]
push 12, 4       [40 12 4]
/                [40 3]
-                [37]       → result 37   (maximum stack size 3)
```

Queue (FIFO)

NOTES

Operations **enqueue** at the rear, **dequeue** from the front — O(1) with a linked list or a circular array. Uses: BFS, scheduling, buffering, level-order traversal.

RULES & FACTS

- **Circular array queue** of size n (indices mod n), with `front` = index of the first element and `rear` = index of the next free slot, one slot kept empty:

```
empty : front == rear
full  : (rear + 1) % n == front        → capacity n − 1
count : (rear − front + n) % n
```

  Without the "wasted slot" trick, front == rear is ambiguous between empty and full, so a separate counter is needed.
- **Queue using two stacks**: push into stack 1; to dequeue, if stack 2 is empty move everything from stack 1 over (reversing order), then pop stack 2. Each element moves at most once → **amortised O(1)** per operation. A **stack using two queues** needs one operation to cost O(n).
- **Deque** allows insertion/deletion at both ends; a **priority queue** removes the highest-priority element (implemented with a heap).

WORKED EXAMPLE

Worked numerical

Circular queue with n = 8, front = 5, rear = 2: count = (2 − 5 + 8) mod 8 = **5** elements (at indices 5, 6, 7, 0, 1).

COMMON TRAP

Remember

BFS uses a **queue**; iterative DFS uses a **stack** (recursion uses the system call stack). Minimum number of stacks needed to simulate a queue = 2; minimum number of queues to simulate a stack = 2.

5 Linked Lists

Node structure and types

FORMULA / WORKING

```
struct node { int data;  struct node *next; };       // singly linked
struct dnode { int data; struct dnode *prev, *next; }; // doubly linked
```

RULES & FACTS

- **Singly**: traverse one way only. **Doubly**: both ways; can delete a given node in O(1). **Circular**: last node points back to the first (no NULL); the traversal condition is `p->next != head`.
- Use a **dummy (sentinel) head node** to avoid special cases for the first element.
- A list with n nodes holds n `next` pointers (singly) or 2n pointers (doubly), plus the head pointer.

Complexity

REFERENCE TABLE

Operation	Singly (head only)	Singly (head + tail)	Doubly
Insert at front	O(1)	O(1)	O(1)
Insert at end	O(n)	O(1)	O(1) with tail
Delete first	O(1)	O(1)	O(1)
Delete last	O(n) (need predecessor)	O(n)	O(1) with tail
Delete a node given a pointer to it	O(n) normally (O(1) trick below)	–	**O(1)**
Search / access k-th	O(n) — binary search is not possible on a linked list in O(log n)		

NOTES

O(1) deletion trick (node is not the last): copy the next node's data into the given node, then unlink and free the next node.

Essential algorithms

FORMULA / RULE

Reverse a list (iterative, O(n) time, O(1) space)

```
prev = NULL; cur = head;
while (cur) {
  nxt = cur->next;
  cur->next = prev;
  prev = cur;  cur = nxt;
}
head = prev;
```

Performs exactly n pointer reversals.

FORMULA / RULE

Middle / cycle (two pointers)

```
slow = fast = head;
while (fast && fast->next) {
  slow = slow->next;
  fast = fast->next->next;   // meets slow iff a cycle
exists
}
// no cycle: slow is the middle
```

Floyd's cycle detection: O(n) time, O(1) space.

RULES & FACTS

- **k-th node from the end**: advance one pointer k steps, then move both until the first hits NULL — one pass.
- **Merge two sorted lists**: O(m + n) time, O(1) extra space. **Merge sort** on a linked list: O(n log n) time (and it is the preferred sort for lists). Insertion sort works on a list in $O(n^2)$.
- Polynomial and large-integer arithmetic, adjacency lists and hash-chaining all rely on linked lists. An **XOR linked list** stores prev ⊕ next in one field to halve the pointer cost.

WORKED EXAMPLE

Worked numerical — recursion on a list

```
void fun(struct node *h) {
    if (!h) return;
    printf("%d ", h->data);
    if (h->next) fun(h->next->next);
    printf("%d ", h->data);
}
List: 1 → 2 → 3 → 4 → 5          fun(head) prints:  1 3 5 5
3 1
fun(1) prints 1, jumps to 3; fun(3) prints 3, jumps to 5;
fun(5) prints 5, next is NULL so no call;
then each frame prints its value again on the way back: 5,
3, 1.
```

WORKED EXAMPLE

Worked numerical — delete alternate nodes

Keep a node, unlink its successor, move to the node after: on 1→2→3→4→5 the loop leaves **1→3→5**; on 1→2→3→4→5→6 it also leaves **1→3→5**.

COMMON TRAP

Array vs linked list

Array: random access O(1), contiguous, cache-friendly, fixed or costly resize, insertion O(n). Linked list: insertion/deletion O(1) once the position is known, no resize cost, extra pointer memory per node, no random access.

6 Trees, BST & AVL

Binary tree facts

NOTES

Take the root at **level 0** and the height of a single node as 0 (height = number of edges on the longest root–leaf path). Be careful: some problems count levels from 1 — read the convention.

RULES & FACTS

- Maximum nodes at level i = 2^i. Maximum nodes in a tree of height h = $\mathbf{2^{h+1} - 1}$ (h = 3 → 15). Minimum height of n nodes = $\lfloor \log_2 n \rfloor$.
- A binary tree with n nodes has **n − 1** edges and **n + 1** NULL child pointers.
- In any binary tree: (leaves) = (nodes with 2 children) + 1.
- For a tree in which every internal node has exactly k children: with i internal nodes, n = k·i + 1 nodes and (k−1)·i + 1 leaves. With k = 3 and i = 10 → **21 leaves**. A full binary tree with i internal nodes has i + 1 leaves.
- Number of structurally different binary trees with n nodes = Catalan(n) = C(2n, n)/(n+1): 1, 2, 5, 14, 42 for n = 1…5. The same number of distinct BSTs can be built on n distinct keys: **4 keys → 14**.
- Complete tree in an array (0-based): children of i are 2i+1 and 2i+2, parent is $\lfloor (i-1)/2 \rfloor$ (1-based: 2i, 2i+1, $\lfloor i/2 \rfloor$).

Traversals

REFERENCE TABLE

Traversal	Order	Typical use
Inorder	Left, Node, Right	sorted output of a BST
Preorder	Node, Left, Right	copy a tree; prefix expression
Postorder	Left, Right, Node	delete a tree; postfix expression; evaluate expression trees
Level order	level by level (queue)	BFS, shortest depth

RULES & FACTS

- All recursive traversals take O(n) time and O(h) stack space. Morris traversal does inorder in O(1) space using threads.
- **Unique reconstruction**: inorder + preorder, or inorder + postorder, determine a binary tree uniquely. **Preorder + postorder do not** (unless every node has 0 or 2 children). The first preorder element (or last postorder element) is the root; split the inorder sequence around it.

Binary search tree (BST)

NOTES

Left subtree keys < node < right subtree keys. Search, insert and delete cost **O(h)**: O(log n) when balanced, **O(n)** when skewed (e.g. keys inserted in sorted order). The **inorder traversal of a BST is sorted**.

Deletion: (1) leaf → remove; (2) one child → replace by that child; (3) two children → replace the key by its **inorder successor** (minimum of the right subtree) or predecessor, then delete that node.

WORKED EXAMPLE

Worked numerical — build, traverse, delete

Insert `50, 30, 70, 20, 40, 60, 80, 35` into an empty BST.

```
        50
      /    \
    30      70
   /  \    /  \
  20  40  60  80
    /
   35
Inorder  : 20 30 35 40 50 60 70 80        (sorted)
Preorder : 50 30 20 40 35 70 60 80
Postorder: 20 35 40 30 60 80 70 50        height = 3

Delete 30 (two children): successor = min of right subtree
{40, 35} = 35 → 35 replaces 30.
   Preorder now: 50 35 20 40 70 60 80
Delete 50 (root, two children): successor = min of {70, 60,
80} = 60 → 60 becomes the root.
   Preorder now: 60 35 20 40 70 80
```

AVL tree (height-balanced)

NOTES

A BST in which the **balance factor** (height of left − height of right) of every node is −1, 0 or +1. After an insert or delete, rebalance with rotations at the lowest unbalanced node:

REFERENCE TABLE

Case	Cause	Fix
LL	insertion into left subtree of left child	one right rotation
RR	insertion into right subtree of right child	one left rotation
LR	insertion into right subtree of left child	left rotation on the child, then right rotation
RL	insertion into left subtree of right child	right rotation on the child, then left rotation

RULES & FACTS

- Height is always O(log n) ($\leq 1.44 \log_2 n$), so search/insert/delete are **O(log n)**. Insertion needs at most **one** (single or double) rotation; deletion may need O(log n) rotations.
- **Minimum nodes** of an AVL tree of height h: N(h) = N(h−1) + N(h−2) + 1 with N(0) = 1, N(1) = 2: **1, 2, 4, 7, 12, 20, 33, 54** for h = 0…7. So height 4 needs at least **12** nodes.

WORKED EXAMPLE

Worked numerical — AVL insertions

Insert `10, 20, 30, 40, 50, 25`:

```
10, 20, 30   → right-right imbalance at 10        → left
rotation          root 20 (10, 30)
40           → fits
50           → right-right imbalance at 30        → left
rotation          20 (10, 40(30, 50))
25           → under 30's left; imbalance at 20 is right-
left → double rotation
Final tree:      30
                /    \
              20      40
             /  \       \
           10   25       50          preorder 30 20 10 25 40
50, height 2
```

A plain (unbalanced) BST with the same insertions has height 4 and degenerates to a list-like shape.

7 Heaps, Hashing & Graph Representation

Binary heap

NOTES

A **complete binary tree** stored in an array. **Max-heap**: every parent ≥ its children (the maximum is at the root); **min-heap**: every parent ≤ its children. With 1-based indexing: left child 2i, right child 2i+1, parent ⌊i/2⌋. A heap with n nodes has height ⌊$\log_2$ n⌋ and **⌈n/2⌉ leaves** (the last ⌈n/2⌉ array positions).

REFERENCE TABLE

Operation	Time	How
Find max (min-heap: min)	O(1)	root
Insert	O(log n)	place at the end, **sift up** while larger than the parent
Delete max	O(log n)	move the last element to the root, **sift down** (swap with the larger child)
Build heap from n elements	**O(n)**	sift down every internal node from ⌊n/2⌋ down to 1 (inserting one by one costs O(n log n))
Heapsort	O(n log n)	build max-heap, then repeatedly swap the root to the end and sift down; in place, not stable
Search for an arbitrary key	O(n)	a heap is not ordered across subtrees
Find the minimum in a max-heap	O(n)	it is one of the leaves

WORKED EXAMPLE

Worked numerical — bottom-up build and one insertion

```
Build a max-heap from [4, 10, 3, 5, 1]   (n = 5, internal
nodes at positions 2 and 1)
 sift-down position 2 (10): children 5, 1 → fine
 sift-down position 1 (4):  swap with 10 → [10, 4, 3, 5, 1];
4 then swaps with 5 → [10, 5, 3, 4, 1]
 Result: [10, 5, 3, 4, 1]   (2 swaps)

Insert 14 into the max-heap [15, 10, 12, 8, 9, 4, 3]:
 append → [15, 10, 12, 8, 9, 4, 3, 14];  14 > parent 8 →
swap;  14 > parent 10 → swap;  14 < 15 stop
 Result: [15, 14, 12, 10, 9, 4, 3, 8]   (2 swaps)

Delete max from [20, 15, 12, 8, 9, 4, 3]: root ← last (3) →
sift down: 3 ↔ 15, then 3 ↔ 9 → [15, 9, 12, 8, 3, 4]
```

FORMULA / RULE

Worked numerical — inserting 1, 2, 3, 4, 5 one by one into a max-heap

Each new key is larger than everything before, so it climbs to the root: swaps = 0 + 1 + 1 + 2 + 2 = **6**, final array **[5, 4, 2, 1, 3]**. Building the same heap bottom-up from [1,2,3,4,5] needs fewer swaps — one reason build-heap is O(n) rather than O(n log n).

Hashing

NOTES

A hash function h(k) maps a key to a slot in a table of size m. With n keys the **load factor** is α = n / m. A **collision** happens when two keys hash to the same slot.

REFERENCE TABLE

Method	How collisions are handled	Notes
Chaining	each slot holds a linked list	α can exceed 1; average search ≈ 1 + α; worst O(n)
Linear probing	try h(k)+1, +2, … (mod m)	simple, cache friendly; suffers **primary clustering**
Quadratic probing	try h(k)+1², +2², …	avoids primary clustering; may not visit every slot
Double hashing	try $h_1(k) + i \cdot h_2(k)$	closest to uniform hashing; $h_2(k)$ must be non-zero and coprime to m

RULES & FACTS

- **Open addressing** stores everything in the table, so n ≤ m (α ≤ 1). Expected probes for an unsuccessful search ≈ 1/(1 − α). Deleting a key must leave a **tombstone** marker, otherwise later searches stop too early.
- Good hash functions spread keys uniformly; the **division method** h(k) = k mod m works best with m prime and not close to a power of 2.
- Average search/insert/delete is **O(1)**; the worst case is O(n). Hash tables do not support ordered operations (min, range queries) efficiently.

WORKED EXAMPLE

Worked numerical — collision resolution (m = 10, h(k) = k mod 10)

Insert `31, 12, 41, 22, 32, 13`.

```
Linear probing:    31→1   12→2   41→1✗2✗→3   22→2✗3✗→4
32→2✗3✗4✗→5   13→3✗4✗5✗→6
   table: [_, 31, 12, 41, 22, 32, 13, _, _, _]       probes:
1, 1, 3, 3, 4, 4  (total 16)
Quadratic (+i²):   table: [_, 31, 12, 22, 13, 41, 32, _, _,
_]      (41 lands at slot 1+4 = 5)
Chaining:          slot 1: 31 → 41     slot 2: 12 → 22 → 32
slot 3: 13
```

Note how linear probing builds one long cluster (slots 1–6) — new keys hashing anywhere into it must probe to its end.

WORKED EXAMPLE

Worked numerical — load factor

1000 keys in a chained table of 200 slots: α = 1000 / 200 = **5** — the average chain holds 5 keys.

Graph representations

REFERENCE TABLE

	Adjacency matrix	Adjacency list
Space	$O(V^2)$	O(V + E)
Is (u, v) an edge?	O(1)	O(deg u)
List all neighbours of u	O(V)	O(deg u)
BFS / DFS of the whole graph	$O(V^2)$	**O(V + E)**
Best for	dense graphs	sparse graphs

RULES & FACTS

- Undirected graph → symmetric matrix; the sum of all matrix entries = 2E. Matrix powers: entry (i, j) of A^k counts walks of length k.
- **BFS** uses a queue and visits by increasing distance; **DFS** uses a stack (or recursion).

WORKED EXAMPLE

Worked numerical — traversal order

Graph with edges 0–1, 0–2, 1–3, 1–4, 2–5, 4–5 (neighbours visited in increasing order), starting at 0:

```
BFS order: 0 1 2 3 4 5          DFS order: 0 1 3 4 5 2
Edges = 6; adjacency matrix has 36 cells; adjacency lists
hold 12 entries.
```

Complexity cheat sheet

REFERENCE TABLE

Structure	Search	Insert	Delete
Unsorted array	O(n)	O(1) at end	O(n)
Sorted array	O(log n)	O(n)	O(n)
Linked list	O(n)	O(1) at known position	O(1) at known position
BST (average / worst)	O(log n) / O(n)	O(log n) / O(n)	O(log n) / O(n)
AVL tree	O(log n)	O(log n)	O(log n)
Binary heap	O(n)	O(log n)	O(log n) (max / min)
Hash table (average / worst)	O(1) / O(n)	O(1) / O(n)	O(1) / O(n)

8 Problem → Standard Method Playbook

1. Pointer declarators: array of pointers vs pointer to array

METHOD

Method: start at the identifier and read the declaration outward, respecting parentheses. `int *p[5]` means an **array of 5 pointers to int**; `int (*p)[5]` means a **pointer to an array of 5 int**. For function pointers, parentheses around `*name` are essential.

WORKED EXAMPLE

Worked problem — declaration parsing

Q: What is `int (*p)[10]`?

METHOD NOTES

Answer: `p` is a pointer to an array of 10 integers. Thus `p+1` advances by `10*sizeof(int)` bytes, not by one integer.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- For an array a with n elements, valid indices are 0 through n-1.
- A binary tree with height h has at most 2^{h+1}-1 nodes when root height is 0.

LAST-MINUTE · MUST REMEMBER

- In C, arrays are contiguous objects; pointer arithmetic is scaled by the pointed-to type.
- String literals are not modifiable; char arrays initialised from literals are distinct from pointers to string literals.
- Call-by-value passes a value; C does not have native call-by-reference, but pointers can be used to modify caller-visible objects.
- Recursion needs a correct base case and progress toward it; analyse recurrence/stack depth separately from total work.
- Stack follows LIFO; queue follows FIFO.
- A linked list provides dynamic node-based storage but loses direct O(1) random indexing.
- BST inorder traversal produces keys in sorted order when the BST invariant holds.
- AVL trees maintain a balance-factor constraint, keeping height O(log n).
- A binary heap is a complete binary tree satisfying heap order; build-heap can be done in O(n).
- Adjacency matrix uses $\Theta(V^2)$ space; adjacency lists use $\Theta(V+E)$ space for sparse graphs.
- Read C declarations from the identifier outward: int *p[5] is an array of pointers; int (*p)[5] is a pointer to an array.
- Integer promotions happen before the usual arithmetic conversions determine the common operand type.
- Morris inorder traversal temporarily threads predecessor links and uses O(1) auxiliary space.
- Bit-field layout is partly implementation-defined; follow the assumptions stated by the question.
- A heap is a complete tree with heap order; it is not a BST.

LAST-MINUTE · COMMON TRAPS

- Array names usually decay to pointers to the first element in expressions, but an array is not itself a pointer.
- **Heap is not a BST:** heap order does not imply globally sorted left/right subtrees.
- volatile does not provide atomicity or inter-thread synchronization.